Программное обеспечение центров обработки данных

Практическая работа №2

Знакомство с Docker

Docker – это программное обеспечение для автоматизации, развёртывания и управления приложениями в средах с поддержкой контейнеризации (виртуализации на уровне ядра операционной системы). Docker позволяет в любой ОС семейства Linux создавать контейнеры с приложениями и требуемым окружением (библиотеки и т.д.), а затем создавать на базе контейнеров образы, копировать их и развертывать его на другом виртуальном или физическом сервере.

Образ – это шаблон операционной системы, установленных в нем библиотек и приложений, который можно запустить в контейнере Docker.

Контейнер – это изолированная часть ядра операционной системы, в которой может быть запущен образ.

Docker также предоставляет возможность скачивать готовые образы из публичного репозитория Docker Hub.

В данной лабораторной работе предлагается выполнить пост-установочную настройку Docker и познакомится с основными операциями с контейнерами.

Подготовка к практической работе

1. Запустите созданную в рамках Практической работы №1 виртуальную машину с Ubuntu и установленным Docker.
2. После загрузки Ubuntu войдите в систему и запустите терминал (**Ctrl+Alt+T** или поиском по приложениям – **Terminal**).

Управление пользователями Docker

По умолчанию после установки Docker запускается от имени пользователя root и другие пользователи имеют возможность управления им при помощи команды sudo, как было сделано в конце первой практики:

```
sudo docker run hello-world
```

Чтобы иметь возможность использовать Docker от лица другого пользователя (например, того, под которым вы вошли в систему) необходимо выполнить следующие действия:

3. В терминале создайте группу для пользователей, имеющих возможность управлять Docker:

```
sudo groupadd docker
```

и нажмите **Enter**. Введите пароль пользователя, нажмите **Enter**.

Возможно, такая группа уже существует, в этом случае дополнительно ничего делать не нужно, переходите к следующему шагу.

4. После выполнения предыдущей команды добавить нужного пользователя в созданную группу docker. В команде ниже вместо **<user>** нужно написать имя пользователя, которому вы хотите дать права использования Docker (без < >). Вы можете предварительно создать нового пользователя или использовать того, под которым вы вошли в систему (его имя отображается в «шапке» терминала):

```
sudo usermod -aG docker <user>
```

5. Выполните выход из текущего сеанса (Log out) или перезагрузите машину (Power off -> Restart), чтобы применить выполненные настройки.
6. После повторного входа в систему запустите терминал и проверьте, что теперь пользоваться Docker от лица добавленного в группу docker пользователя можно без прав root-пользователя:

```
docker run hello-world
```

Docker Desktop для Mac и Windows

Для операционных систем Windows и Mac OS существует возможность установки графического интерфейса для работы с Docker. Более подробную информацию можно найти по ссылке: https://docs.docker.com/desktop/

Использование Docker Hub

Важным преимуществом Docker по сравнению с другими системами виртуализации на уровне ядра операционной системы является возможность

скачивать готовые образы для разных нужд из публичного репозитория Docker Hub, а также сохранять в него свои образы. В частности, образ hello-world, использовавшийся ранее для проверки работоспособности Docker был скачан именно оттуда. Для изучения работы с Docker Hub выполните следующие действия:

1. Запустите терминал и выполните команду:

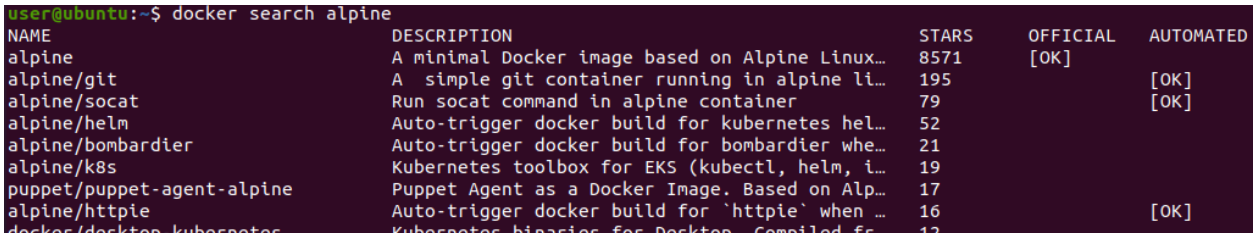
```
docker images
```

Если ранее не было выполнено настройки пользователей, перед указанной выше командой надо написать команду sudo. В терминале появится список загруженных образов, в котором пока будет только загруженный ранее образ **hello-world**.

2. Для того, чтобы найти в Docker Hub нужный вам образ, можно воспользоваться командой search. Например, найдем образ с установленным ЯП python:

```
docker search alpine
```

В терминале появятся все образы, найденные по ключевому слову alpine. В таблице будут приведены из названия, описания, количество запусков. Также в таблице есть столбец, показывающий, является ли этот образ официальным:



3. Выбрав из результатов поиска нужный образ (в данном случае образ с именем ***alpine***) можно скачать его командой:

```
docker pull alpine
```

Скачивать образ отдельным шагом не обязательно, можно сразу написать команду запуска нужного образа в новом контейнере (см. ниже), тогда, если образ с таким именем не будет найден в локальном репозитории, он будет сначала скачан, а потом запущен.

4. После скачивания образа ***alpine*** выполните команду для просмотра всех имеющихся образов:

```
docker images
```

Обратите внимание, что в списке появился новый образ с именем ***alpine***.

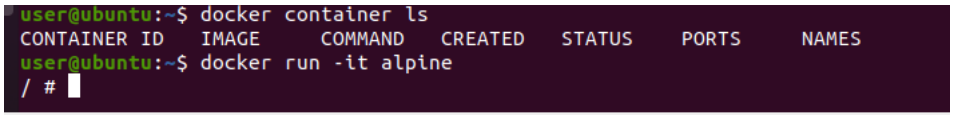
5. Далее выполните команду для просмотра всех запущенных контейнеров (выделенных частей ядра операционной системы, в которых запускается образ):

```
docker container ls
```

6. В выводе предыдущей команды должны отображаться все запущенные контейнеры. К этому моменту в нем не должно быть запущенных контейнеров, поэтому выполните команду для запуска образа ***alpine*** в новом контейнере:

```
docker run -it alpine
```

Ключи i и t позволяют в интерактивном режиме подключиться к командной строке контейнера. После введения этой команды внешний вид командной строки изменится:



Далее в этом контейнере можно выполнить какие-либо действия, например, установить веб-сервер apache:

```
apk update
apk add apache2
```

Результат этих действий останется в запущенном контейнере, поэтому в дальнейшем для работы с этим контейнером необходимо будет запускать именно его, а не новый контейнер с образом, как в команде выше. Для запуска уже имеющегося контейнера можно выполнить команду:

```
docker container start <имя контейнера>
```

Для остановки (выключения) контейнера:

```
docker container stop <имя контейнера>
```

Если в выводе команды ls уже есть запущенный контейнер, можно подключиться к нему при помощи команды attach:

```
docker container attach <имя контейнера>
```

7. Для выхода из командной строки контейнера можно ввести команду exit.

8. Из контейнера со внесенными изменениями можно создать новый образ, например, чтобы запускать его на других серверах. Для создания образа из существующего контейнера необходимо выполнить команду:

```
docker commit -m "Add apache2" <имя контейнера> <имя нового образа>
```

   Созданный образ сохраняется локально, и, если вы не планируете в дальнейшем загружать его на Docker Hub, в качестве имени образа можно выбрать любое осмысленное название. Если же вы собираетесь делиться созданным образом с миром, лучше воспользоваться правилами именования репозиториев и образов https://docs.docker.com/engine/reference/commandline/tag/

9. После создания своего образа убедитесь, что он появился в списке образов:

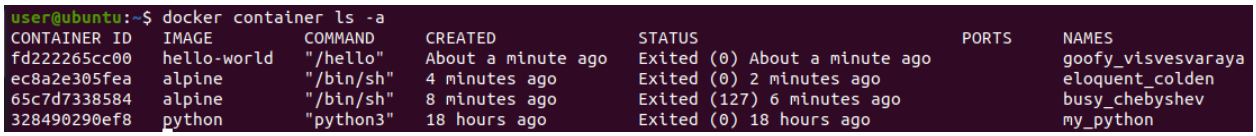
```
docker images
```

Обратите внимание, что размер созданного вами образа больше, чем размер образа ***alpine***, на основе которого он был создан.

10. Для просмотра всех имеющихся контейнеров, не только запущенных в настоящий момент, введите команду:

```
docker container ls -a
```

В выводе этой команды будут представлены все имеющиеся в настоящий момент контейнеры:



Обратите внимание, что к контейнеру можно обращаться либо по его id (первый столбец), или по имени (последний столбец). Имя контейнеру изначально присваивается случайным образом. Переименовать его можно при помощи команды:

```
docker container rename <текущее имя контейнера> <новое имя контейнера>
```

11. Для удаления лишних контейнеров (например, с образом hello-world) используйте команду:

```
docker container rm <имя контейнера>
```

12. Для удаления ненужных образов (например, hello-world) введите команду:

```
docker image rm hello-world
```

13. После завершения работы выключите виртуальную машину закрыв окно и выбрав Suspend или Power off.

Более подробную информацию по шагам в данной Практической работе можно найти по ссылкам:

https://docs.docker.com/engine/install/linux-postinstall/

https://docs.docker.com/engine/reference/commandline/cli/